시작하십시오
Astra Trident

NetApp
November 28, 2023

This PDF was generated from https://docs.netapp.com/ko-kr/trident-2307/trident-get-started/intro.html on November 28, 2023. Always check docs.netapp.com for the latest.

목차

시작하십시오 1
Astra Trident에 대해 자세히 알아보십시오 1
Astra Trident를 위한 빠른 시작 7
요구 사항 8

시작하십시오

Astra Trident에 대해 자세히 알아보십시오

Astra Trident에 대해 자세히 알아보십시오

Astra Trident는 NetApp이 의 일부로 관리하며 완벽한 지원이 제공되는 오픈 소스 프로젝트입니다 "Astra 제품군". CSI(Container Storage Interface)와 같은 업계 표준 인터페이스를 사용하여 컨테이너식 애플리케이션의 지속성 요구 사항을 충족하도록 설계되었습니다.

아스트라(Astra)란?

Astra를 사용하면 퍼블릭 클라우드와 온프레미스 모두에서 Kubernetes에서 실행되는 데이터 기반의 컨테이너 워크로드를 손쉽게 관리, 보호 및 이동할 수 있습니다.

Astra는 Astra Trident를 기반으로 영구 컨테이너 스토리지를 프로비저닝하고 제공합니다. 또한 스냅샷, 백업 및 복원, 활동 로그, 데이터 보호를 위한 활성 클론 복제, 재해/데이터 복구, 데이터 감사, Kubernetes 워크로드를 위한 마이그레이션 사용 사례와 같은 고급 애플리케이션 인식 데이터 관리 기능도 제공합니다.

에 대해 자세히 알아보십시오 "Astra 또는 무료 평가판에 등록하십시오".

Astra Trident란?

Astra Trident를 사용하면 ONTAP(AFF, NetApp FAS, Select, 클라우드, NetApp ONTAP용 Amazon FSx), Element 소프트웨어(NetApp HCI, SolidFire), Azure NetApp Files 서비스 및 Google Cloud 기반 Cloud Volumes Service

Astra Trident는 기본적으로 와 통합되는 CSI(컨테이너 스토리지 인터페이스) 호환 동적 스토리지 오케스트레이터입니다 "쿠버네티스". Astra Trident는 클러스터의 각 작업자 노드에서 단일 컨트롤러 Pod와 노드 Pod로 실행됩니다. 을 참조하십시오 "Astra Trident 아키텍처" 를 참조하십시오.

또한 Astra Trident는 NetApp 스토리지 플랫폼을 위한 Docker 에코시스템과의 직접 통합을 제공합니다. NetApp Docker 볼륨 플러그인(nDVP)은 스토리지 플랫폼에서 Docker 호스트로 스토리지 리소스를 프로비저닝 및 관리할 수 있도록 지원합니다. 을 참조하십시오 "Docker를 위한 Astra Trident 구축" 를 참조하십시오.

> Kubernetes를 처음으로 사용하는 경우라면 에 익숙해야 합니다 "Kubernetes 개념 및 툴".

Astra Trident 시험 구동을 시작하십시오

시험 구동을 실시하려면 "컨테이너식 워크로드용 영구 스토리지를 간편하게 배포 및 복제" 액세스 권한을 요청하십시오. "NetApp 시험 구동" 즉시 사용 가능한 랩 이미지 사용 테스트 드라이브는 3노드 Kubernetes 클러스터와 Astra Trident가 설치 및 구성된 샌드박스 환경을 제공합니다. Astra Trident에 대해 알아보고 해당 기능을 살펴볼 수 있는 좋은 방법입니다.

또 다른 옵션은 입니다 "kubeadm 설치 가이드" Kubernetes에서 제공:

> 운영 환경에서는 이러한 지침에 따라 구축한 Kubernetes 클러스터를 사용하지 마십시오. 운영 가능 클러스터에 대해 배포에서 제공하는 운영 구축 가이드를 사용하십시오.

를 참조하십시오

- "NetApp Astra 제품군"
- "Astra Control Service 문서"
- "Astra Control Center 문서"
- "Astra API 설명서"

Astra Trident 아키텍처

Astra Trident는 클러스터의 각 작업자 노드에서 단일 컨트롤러 Pod와 노드 Pod로 실행됩니다. Astra Trident 볼륨을 마운트하려는 호스트에서 노드 포드가 실행되고 있어야 합니다.

컨트롤러 Pod 및 노드 포드 이해

Astra Trident가 단일 플랫폼으로 구축됩니다 Trident 컨트롤러 Pod 및 하나 이상 Trident 노드 Pod Kubernetes 클러스터에서 표준 Kubernetes_CSI Sidecar Containers_를 사용하여 CSI 플러그인 배포를 간소화합니다. "Kubernetes CSI Sidecar Containers의 약어입니다" Kubernetes 스토리지 커뮤니티에서 유지 관리합니다.

쿠버네티스 "노드 선택기" 및 "관용과 오해" 포드를 특정 노드 또는 기본 노드에서 실행하도록 제한하는 데 사용됩니다. Astra Trident 설치 중에 컨트롤러 및 노드 포드의 노드 선택기 및 허용을 구성할 수 있습니다.

- 컨트롤러 플러그인은 스냅샷 및 크기 조정과 같은 볼륨 프로비저닝 및 관리를 처리합니다.
- 노드 플러그인은 스토리지에 노드를 연결하는 작업을 처리합니다.

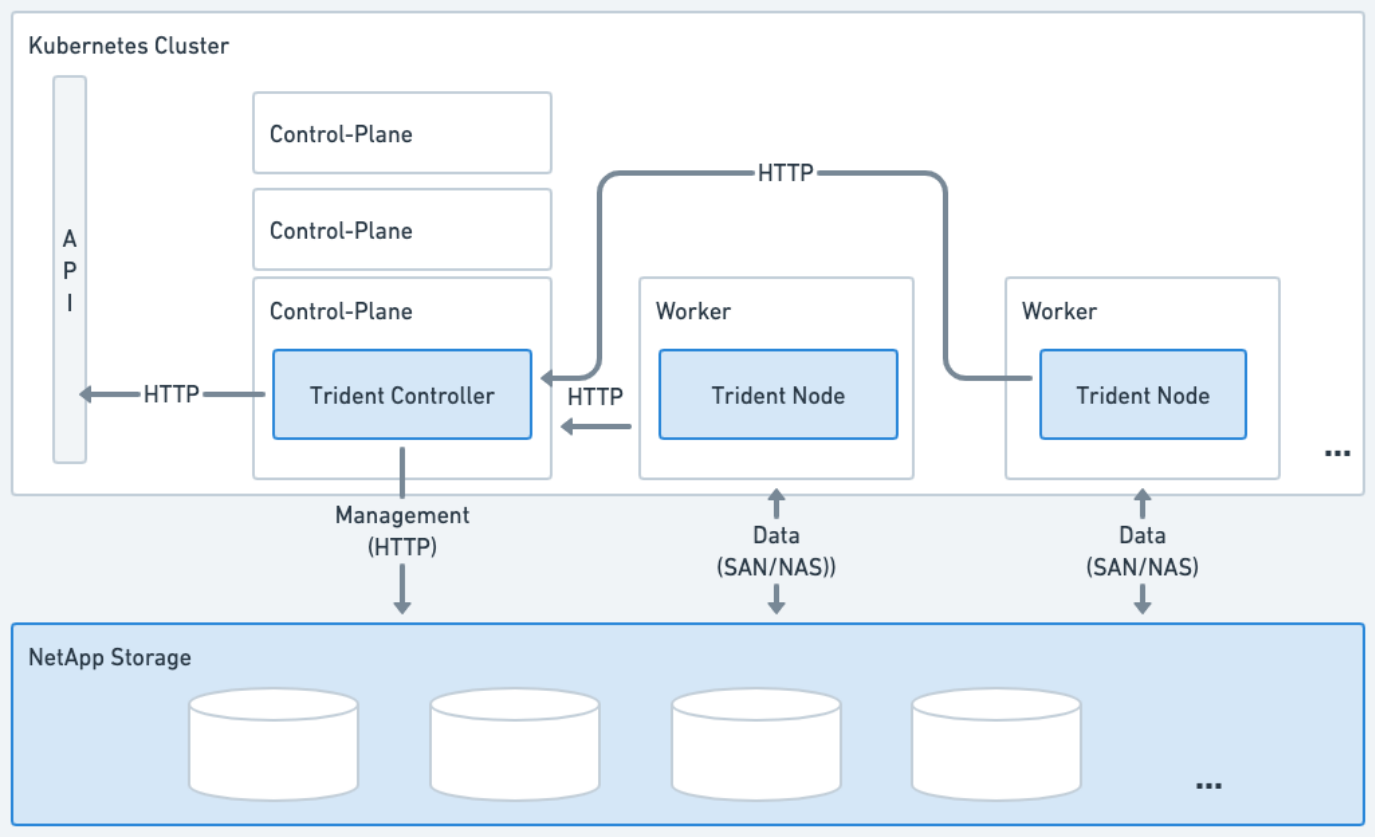


그림 **1. Kubernetes** 클러스터에 배포된 **Astra Trident**

Trident 컨트롤러 Pod

Trident 컨트롤러 Pod는 CSI 컨트롤러 플러그인을 실행하는 단일 Pod입니다.

- NetApp 스토리지에서 볼륨을 프로비저닝하고 관리하는 업무를 담당합니다
- Kubernetes 배포에 의해 관리됩니다
- 설치 매개변수에 따라 컨트롤 플레인 또는 작업자 노드에서 실행할 수 있습니다.

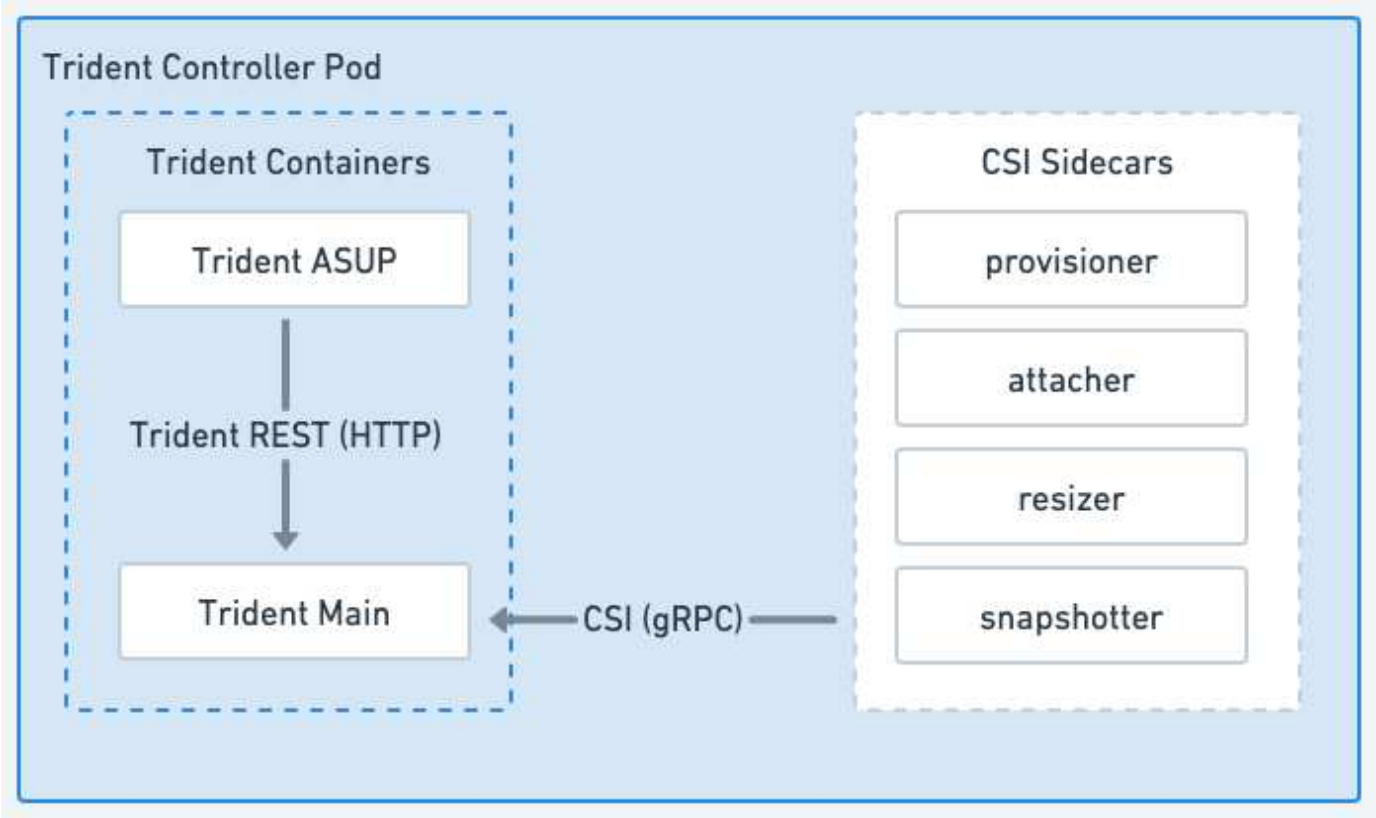


그림 **2. Trident** 컨트롤러 **Pod** 다이어그램

Trident 노드 Pod

Trident 노드 Pod는 CSI 노드 플러그인을 실행하는 권한 있는 Pod입니다.

- 호스트에서 실행 중인 Pod에 대한 스토리지 마운트 및 마운트 해제를 담당합니다
- Kubernetes DemonSet에서 관리합니다
- NetApp 스토리지를 마운트할 모든 노드에서 실행해야 합니다

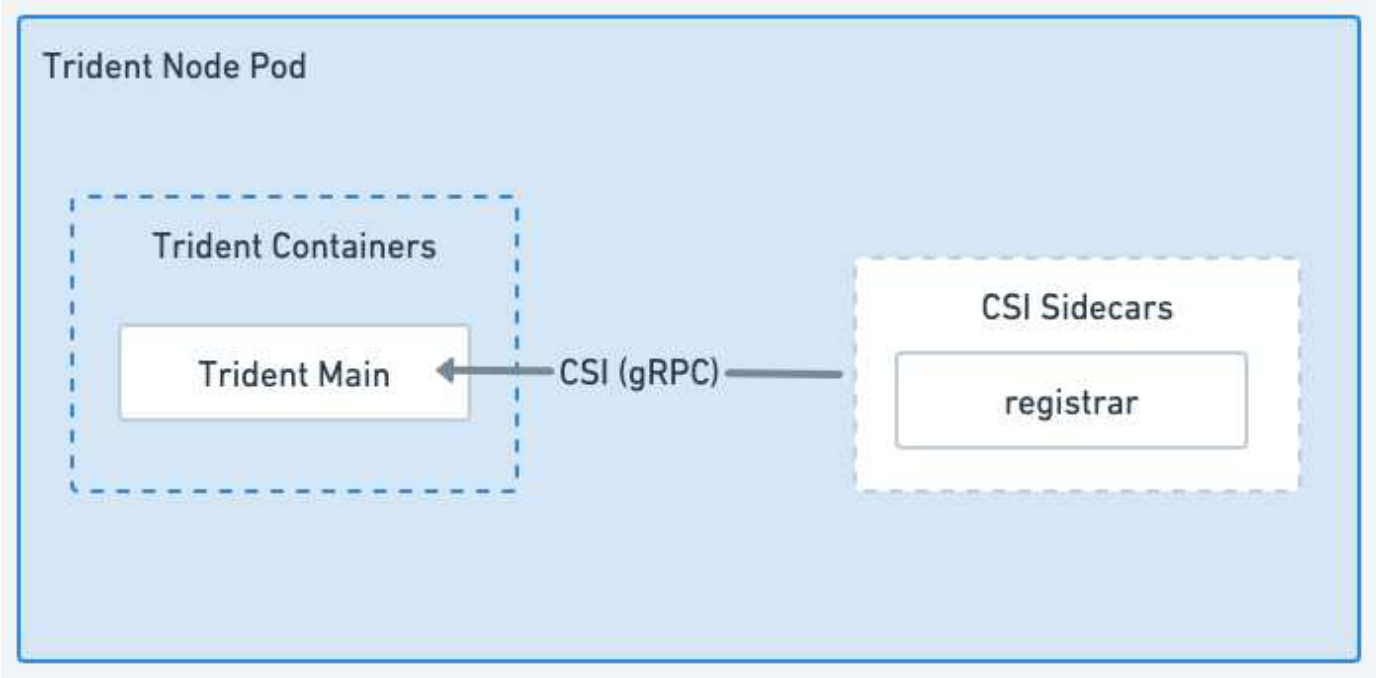


그림 **3. Trident** 노드 **Pod** 다이어그램

지원되는 **Kubernetes** 클러스터 아키텍처

Astra Trident는 다음 Kubernetes 아키텍처에서 지원됩니다.

Kubernetes 클러스터 아키텍처	지원	기본 설치
단일 마스터, 컴퓨팅	예	예
다중 마스터, 컴퓨팅	예	예
마스터, `etcd`컴퓨팅	예	예
마스터, 인프라, 컴퓨팅	예	예

개념

프로비저닝

Astra Trident의 프로비저닝에는 두 가지 주요 단계가 있습니다. 첫 번째 단계에서는 스토리지 클래스를 적절한 백엔드 스토리지 풀 세트와 연결하고 용량 할당 전에 필요한 준비 작업으로 수행됩니다. 두 번째 단계에서는 볼륨 생성 자체를 포함하며 보류 중인 볼륨의 스토리지 클래스와 연결된 스토리지 풀을 선택해야 합니다.

스토리지 클래스 연결

백엔드 스토리지 풀을 스토리지 클래스와 연결하려면 스토리지 클래스의 요청된 속성 및 해당 특성이 모두 필요합니다 `storagePools`, `additionalStoragePools`, 및 `excludeStoragePools` 목록. 스토리지 클래스를 생성할 때 Trident는 각 백엔드에서 제공하는 특성과 풀을 스토리지 클래스에서 요청한 속성과 비교합니다. 스토리지 풀의 속성과

이름이 요청된 모든 속성 및 풀 이름과 일치하면 Astra Trident는 해당 스토리지 클래스에 적합한 스토리지 풀 세트에 해당 스토리지 풀을 추가합니다. 또한 Astra Trident는 에 나열된 모든 스토리지 풀을 추가합니다 `additionalStoragePools` 특성이 요청된 스토리지 클래스의 모든 속성 또는 특성을 충족하지 않는 경우에도 해당 집합에 대해 나열합니다. 를 사용해야 합니다 `excludeStoragePools` 스토리지 클래스에 사용할 스토리지 풀을 재정의하고 제거할 목록입니다. Astra Trident는 새 백엔드를 추가할 때마다 유사한 프로세스를 수행하여 스토리지 풀이 기존 스토리지 클래스의 풀을 충족하는지 확인하고 제외로 표시된 항목을 제거합니다.

볼륨 생성

그런 다음 Astra Trident는 스토리지 클래스와 스토리지 풀 간의 연결을 사용하여 볼륨을 프로비저닝할 위치를 결정합니다. 볼륨을 생성할 때 Astra Trident는 먼저 해당 볼륨의 스토리지 클래스에 대한 스토리지 풀 세트를 가져옵니다. 볼륨의 프로토콜을 지정한 경우 Astra Trident는 요청된 프로토콜을 제공할 수 없는 스토리지 풀을 제거합니다(예: NetApp HCI/SolidFire 백엔드는 파일 기반 볼륨을 제공할 수 없고 ONTAP NAS 백엔드는 블록 기반 볼륨을 제공할 수 없음). Astra Trident는 이 결과 집합의 순서를 무작위로 생성하여 볼륨을 균일하게 분산시킨 다음 이를 반복함으로써 각 스토리지 풀에서 볼륨을 차례로 프로비저닝합니다. 이 오류가 1에서 성공하면 프로세스에서 발생한 모든 오류를 로깅하여 성공적으로 반환됩니다. Astra Trident는 * 요청된 스토리지 클래스 및 프로토콜에 대해 사용 가능한 모든 * 스토리지 풀을 프로비저닝하지 못한 경우에만 실패 * 를 반환합니다.

볼륨 스냅숏

Astra Trident가 드라이버를 위한 볼륨 스냅샷 생성을 어떻게 처리하는지 자세히 알아보십시오.

볼륨 스냅샷 생성에 대해 자세히 알아보십시오

- 의 경우 `ontap-nas`, `ontap-san`, `gcp-cvs`, 및 `azure-netapp-files` 드라이버, 각 PV(영구 볼륨)는 FlexVol에 매핑됩니다. 따라서 볼륨 스냅샷이 NetApp 스냅샷으로 생성됩니다. NetApp 스냅샷 기술은 경쟁 스냅샷 기술보다 더 높은 안정성, 확장성, 복구 가능성 및 성능을 제공합니다. 이러한 스냅샷 복사본은 생성하는 데 필요한 시간과 스토리지 공간 모두에서 매우 효율적입니다.
- 의 경우 `ontap-nas-flexgroup` 드라이버, 각 PV(영구 볼륨)는 FlexGroup에 매핑됩니다. 따라서 볼륨 스냅샷이 NetApp FlexGroup 스냅샷으로 생성됩니다. NetApp 스냅샷 기술은 경쟁 스냅샷 기술보다 더 높은 안정성, 확장성, 복구 가능성 및 성능을 제공합니다. 이러한 스냅샷 복사본은 생성하는 데 필요한 시간과 스토리지 공간 모두에서 매우 효율적입니다.
- 의 경우 `ontap-san-economy` 드라이버, PVS는 공유 FlexVol에 생성된 LUN에 매핑됩니다. PVS의 볼륨 스냅샷은 연결된 LUN의 FlexClone을 수행하여 구현됩니다. ONTAP FlexClone 기술을 사용하면 가장 큰 데이터 세트의 복사본을 거의 즉시 생성할 수 있습니다. 복사본은 데이터 블록을 부모님과 공유하므로 메타데이터에 필요한 것만 빼고 스토리지를 사용하지 않습니다.
- 의 경우 `solidfire-san` 드라이버, 각 PV는 NetApp Element 소프트웨어/NetApp HCI 클러스터에서 생성된 LUN에 매핑됩니다. 볼륨 스냅샷은 기본 LUN의 요소 스냅샷으로 표시됩니다. 이러한 스냅샷은 특정 시점 복제본이며 소량의 시스템 리소스와 공간만 차지합니다.
- 를 사용하여 작업하는 경우 `ontap-nas` 및 `ontap-san` 드라이버 및 ONTAP 스냅샷은 FlexVol의 시점 복사본으로 FlexVol 자체에서 공간을 사용합니다. 따라서 스냅샷이 생성/예약될 때 시간이 경과함에 따라 볼륨의 쓰기 가능한 공간이 줄어들 수 있습니다. 이 문제를 해결하는 간단한 방법 중 하나는 Kubernetes를 통해 크기를 조정하여 볼륨을 늘리는 것입니다. 또 다른 옵션은 더 이상 필요하지 않은 스냅샷을 삭제하는 것입니다. Kubernetes를 통해 생성된 VolumeSnapshot이 삭제되면 Astra Trident가 연결된 ONTAP 스냅샷을 삭제합니다. Kubernetes를 통해 생성되지 않은 ONTAP 스냅샷도 삭제할 수 있습니다.

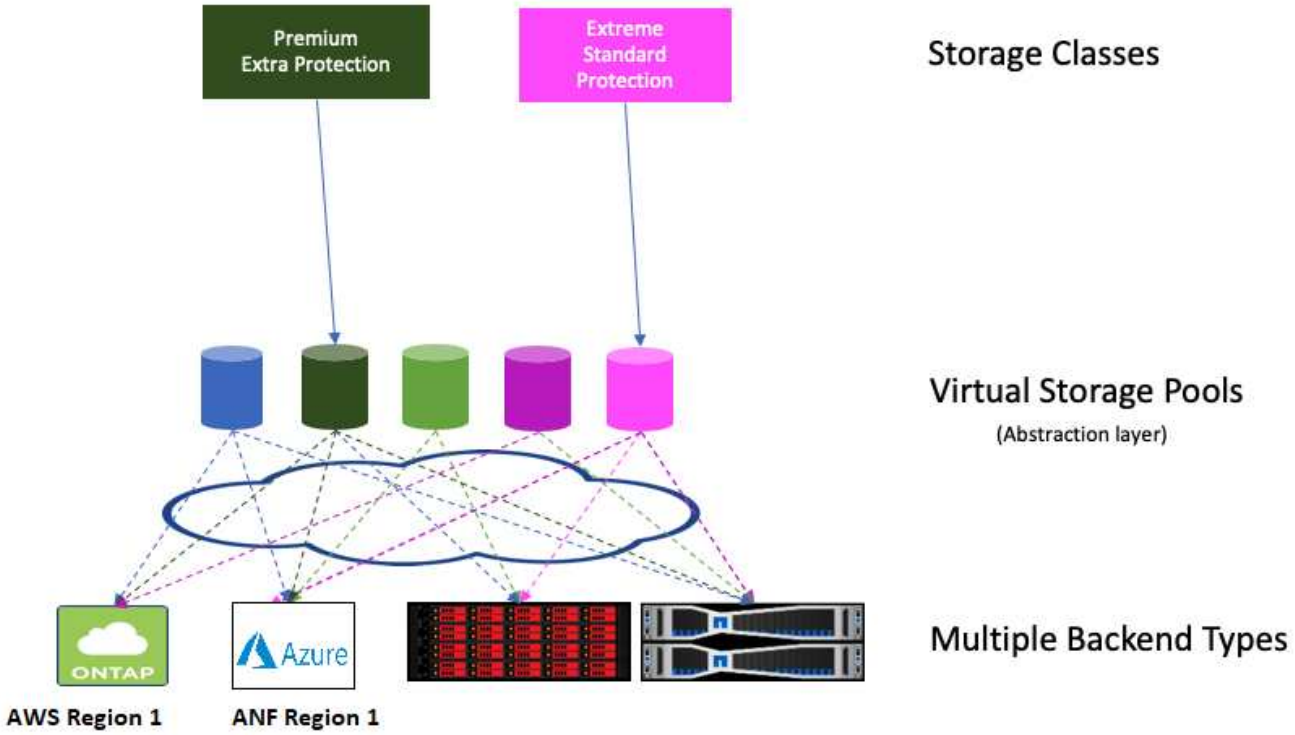
Astra Trident를 사용하면 VolumeSnapshots을 사용하여 새 PVS를 생성할 수 있습니다. 이러한 스냅샷에서 PVS를 생성하는 작업은 지원되는 ONTAP 및 CVS 백엔드에 FlexClone 기술을 사용하여 수행됩니다. 스냅샷에서 PV를 생성할 때 백업 볼륨은 스냅샷의 상위 볼륨의 FlexClone입니다. 를 클릭합니다 `solidfire-san` 드라이버는 Element 소프트웨어 볼륨 클론을 사용하여 스냅샷에서 PVS를 생성합니다. 여기서 Element 스냅샷으로부터 클론을 생성합니다.

가상 풀

가상 풀은 Astra Trident 스토리지 백엔드와 Kubernetes 간의 추상화 계층을 제공합니다 `StorageClasses`. 관리자는 을 만들지 않고 각 백엔드에 대한 위치, 성능 및 보호와 같은 측면을 일반적인 백엔드 방식으로 정의할 수 있습니다 `StorageClass` 원하는 기준을 충족하는 데 사용할 물리적 백엔드, 백엔드 풀 또는 백엔드 유형을 지정합니다.

가상 풀에 대해 알아보십시오

스토리지 관리자는 Astra Trident 백엔드의 가상 풀을 JSON 또는 YAML 정의 파일로 정의할 수 있습니다.



가상 풀 목록 외부에서 지정된 모든 측면은 백엔드에 대해 전역적이며 모든 가상 풀에 적용되지만, 각 가상 풀은 하나 이상의 측면을 개별적으로 지정할 수 있습니다(백엔드-글로벌 측면 재정의).

- 가상 풀을 정의할 때 백엔드 정의에서 기존 가상 풀의 순서를 재정렬하지 마십시오.
- 기존 가상 풀에 대한 속성을 수정하지 않는 것이 좋습니다. 변경하려면 새 가상 풀을 정의해야 합니다.

대부분의 측면은 백엔드 관련 용어로 지정됩니다. 결정적으로, Aspect 값은 백엔드의 드라이버 외부에 노출되지 않으며 에서 일치시킬 수 없습니다 `StorageClasses`. 대신 관리자는 각 가상 풀에 대해 하나 이상의 레이블을 정의합니다. 각 레이블은 키, 즉 값 쌍이며 레이블은 고유한 백엔드에서 공통일 수 있습니다. 측면과 마찬가지로 레이블을 풀별로 지정하거나 백엔드에 대해 전역으로 지정할 수 있습니다. 미리 정의된 이름과 값이 있는 측면과 달리 관리자는 필요에 따라 레이블 키와 값을 정의할 수 있습니다. 편의를 위해 스토리지 관리자는 가상 풀 및 그룹 볼륨별로 레이블을 레이블별로 정의할 수 있습니다.

A `StorageClass` 선택기 매개 변수 내의 레이블을 참조하여 사용할 가상 풀을 식별합니다. 가상 풀 선택기는 다음

연산자를 지원합니다.

운영자	예	풀의 레이블 값은 다음과 같아야 합니다.
`=`	성능 = 프리미엄	일치
`!=`	성능!=최고	일치하지 않습니다
`in`	위치(동쪽, 서쪽)	값 집합에 있어야 합니다
`notin`	성능 노트(실버, 브론즈)	값 집합에 있지 않습니다
`<key>`	보호	모든 값과 함께 존재합니다
`!<key>`	! 보호	존재하지 않습니다

볼륨 액세스 그룹

Astra Trident가 사용하는 방법에 대해 자세히 알아보십시오 "볼륨 액세스 그룹".

> CHAP를 사용하는 경우 이 섹션을 무시하십시오. CHAP는 관리를 단순화하고 아래 설명된 배율 제한을 피하는 것이 좋습니다. 또한 CSI 모드에서 Astra Trident를 사용하는 경우 이 섹션을 무시할 수 있습니다. Astra Trident는 향상된 CSI 구축 기능으로 설치된 경우 CHAP를 사용합니다.

볼륨 액세스 그룹에 대해 알아보십시오

Astra Trident는 볼륨 액세스 그룹을 사용하여 프로비저닝하는 볼륨에 대한 액세스를 제어할 수 있습니다. CHAP가 해제되어 있으면 에서 라는 액세스 그룹을 찾을 것으로 예상합니다 `trident` 구성에 하나 이상의 액세스 그룹 ID를 지정하지 않은 경우

Astra Trident는 새 볼륨을 구성된 액세스 그룹과 연결하지만 액세스 그룹 자체를 생성하거나 관리하지 않습니다. 액세스 그룹은 스토리지 백엔드가 Astra Trident에 추가되기 전에 존재해야 하며, Kubernetes 클러스터의 모든 노드에 있는 iSCSI IQN을 포함해야 하며, 이 백엔드에서 프로비저닝한 볼륨을 잠재적으로 마운트할 수 있습니다. 대부분의 설치에서는 클러스터의 모든 작업자 노드를 포함합니다.

64개 이상의 노드가 있는 Kubernetes 클러스터의 경우 다중 액세스 그룹을 사용해야 합니다. 각 액세스 그룹에는 최대 64개의 IQN이 포함될 수 있으며 각 볼륨은 4개의 액세스 그룹에 속할 수 있습니다. 최대 4개의 액세스 그룹이 구성되어 있는 경우, 클러스터의 모든 노드에서 최대 256개의 노드 크기로 모든 볼륨에 액세스할 수 있습니다. 볼륨 액세스 그룹에 대한 최신 제한은 를 참조하십시오 "여기".

기본값을 사용하는 구성에서 구성을 수정하는 경우 `trident` 다른 사람을 사용하는 그룹에 대한 액세스 그룹에는 의 ID도 포함됩니다 `trident` 목록에 있는 액세스 그룹.

Astra Trident를 위한 빠른 시작

Astra Trident를 설치하고 몇 단계로 스토리지 리소스 관리를 시작할 수 있습니다. 시작하기 전에 를 검토하십시오 "Astra Trident 요구사항".

> Docker의 경우 를 참조하십시오 "Docker를 위한 Astra Trident".

1. **Astra Trident**를 설치합니다

Astra Trident는 다양한 환경 및 조직에 최적화된 다양한 설치 방법 및 모드를 제공합니다.

"Astra Trident를 설치합니다"

2 작업자 노드를 준비합니다

Kubernetes 클러스터의 모든 작업자 노드는 Pod용으로 프로비저닝된 볼륨을 마운트할 수 있어야 합니다.

"작업자 노드를 준비합니다"

3 백엔드를 생성합니다

백엔드는 Astra Trident와 스토리지 시스템 간의 관계를 정의합니다. Astra Trident가 스토리지 시스템과 통신하는 방법과 Astra Trident가 스토리지 시스템에서 볼륨을 프로비저닝하는 방법을 알려줍니다.

"백엔드를 구성합니다" 확인하십시오

4 **Kubernetes StorageClass**를 생성합니다

Kubernetes StorageClass 객체는 Astra Trident를 프로비저닝으로 지정하고, 스토리지 클래스를 생성하여 사용자 지정 가능한 속성으로 볼륨을 프로비저닝할 수 있습니다. Astra Trident는 Astra Trident provisioner를 지정하는 Kubernetes 오브젝트에 맞는 스토리지 클래스를 생성합니다.

"스토리지 클래스를 생성합니다"

5 볼륨을 프로비저닝합니다

A_PersistentVolume_(PV)은 Kubernetes 클러스터에서 클러스터 관리자가 프로비저닝한 물리적 스토리지 리소스입니다. PVC(PersistentVolumeClaim_)는 클러스터의 PersistentVolume에 대한 액세스 요청입니다.

구성된 Kubernetes StorageClass를 사용하여 PV에 대한 액세스를 요청하는 PersistentVolume(PV) 및 PersistentVolumeClaim(PVC)을 생성합니다. 그런 다음 PV를 포드에 장착할 수 있습니다.

"볼륨을 프로비저닝합니다"

다음 단계

이제 백엔드를 추가하고, 스토리지 클래스를 관리하고, 백엔드를 관리하고, 볼륨 작업을 수행할 수 있습니다.

요구 사항

Astra Trident를 설치하기 전에 이러한 일반 시스템 요구 사항을 검토해야 합니다. 특정 백엔드에 추가 요구 사항이 있을 수 있습니다.

Astra Trident에 대한 중요 정보

- Astra Trident * 에 대한 다음 중요 정보를 읽어야 합니다

** 중요 정보 Astra Trident **

- 이제 Trident에서 Kubernetes 1.27이 지원됩니다. Kubernetes를 업그레이드하기 전에 Trident를 업그레이드하십시오.
- Astra Trident는 SAN 환경에서 다중 경로 구성을 엄격하게 사용하며 권장 값은 입니다 `find_multipaths: no` 다중 경로 .conf 파일

  비 경로 다중화 구성 또는 의 사용 `find_multipaths: yes` 또는 `find_multipaths: smart` multipath.conf 파일의 값으로 인해 마운트 오류가 발생합니다. Trident에서 의 사용을 권장했습니다 `find_multipaths: no` 21.07 릴리스 이후.

지원되는 프런트엔드(오케스트레이터)

Astra Trident는 다음을 비롯한 여러 컨테이너 엔진과 오케스트레이터가 지원됩니다.

- Anthos 온프레미스(VMware) 및 Anthos의 Bare Metal 1.12
- Kubernetes 1.22-1.27
- Mirantis Kubernetes 엔진 3.5
- OpenShift 4.10-4.13

Trident 연산자는 다음 릴리즈에서 지원됩니다.

- Anthos 온프레미스(VMware) 및 Anthos의 Bare Metal 1.12
- Kubernetes 1.22-1.27
- OpenShift 4.10-4.13

Astra Trident는 GKE(Google Kubernetes Engine), EKS(Amazon Elastic Kubernetes Service), AKS(Azure Kubernetes Service), Rancher, VMware Tanzu Portfolio를 비롯한 기타 완전 관리형 및 자체 관리 Kubernetes 오퍼링과도 작동합니다.

> Astra Trident가 설치된 Kubernetes 클러스터를 1.24에서 1.25 이상으로 업그레이드하기 전에 을 참조하십시오 "Helm 설치를 업그레이드합니다".

지원되는 백엔드(스토리지)

Astra Trident를 사용하려면 다음 중 하나 이상의 지원되는 백엔드가 필요합니다.

- NetApp ONTAP용 Amazon FSx
- Azure NetApp Files
- Cloud Volumes ONTAP
- GCP용 Cloud Volumes Service
- FAS/AFF/9.5 이상을 선택합니다
- NetApp All SAN 어레이(ASA)
- NetApp HCI/Element 소프트웨어 11 이상

피처 요구 사항

아래 표에는 Astra Trident의 이번 릴리즈와 함께 사용할 수 있는 기능과 지원하는 Kubernetes 버전이 요약되어 있습니다.

피처	**Kubernetes** 버전	기능 게이트가 필요합니까**?**
아스트라 트리덴트	1.22-1.27	아니요
볼륨 스냅샷	1.22-1.27	아니요
체적 스냅샷의 PVC	1.22-1.27	아니요
iSCSI PV 크기 조정	1.22-1.27	아니요
ONTAP 양방향 CHAP	1.22-1.27	아니요
동적 내보내기 정책	1.22-1.27	아니요
Trident 연산자	1.22-1.27	아니요
CSI 토폴로지	1.22-1.27	아니요

호스트 운영 체제를 테스트했습니다

Astra Trident가 공식적으로 특정 운영 체제를 지원하지 않지만 다음과 같은 운영 체제가 작동하는 것으로 알려져 있습니다.

- OpenShift Container Platform(AMD64 및 ARM64)에서 지원하는 RedHat CoreOS(RHCOS) 버전
- RHEL 8+(AMD64 및 ARM64)
- Ubuntu 22.04 이상(AMD64 및 ARM64)
- Windows Server 2019(AMD64)

기본적으로 Astra Trident는 컨테이너에서 실행되므로 모든 Linux 작업자에서 실행됩니다. 그러나 이러한 작업자들은 표준 NFS 클라이언트 또는 iSCSI 이니시에이터를 사용하여 Astra Trident가 제공하는 볼륨을 사용 중인 백엔드에 따라 마운트할 수 있어야 합니다.

를 클릭합니다 `tridentctl` 이 유틸리티는 이러한 Linux 배포판에서도 실행됩니다.

호스트 구성

Kubernetes 클러스터의 모든 작업자 노드는 Pod용으로 프로비저닝된 볼륨을 마운트할 수 있어야 합니다. 작업자 노드를 준비하려면 드라이버 선택에 따라 NFS 또는 iSCSI 도구를 설치해야 합니다.

"작업자 노드를 준비합니다"

스토리지 시스템 구성

Astra Trident는 백엔드 구성에서 사용하기 전에 스토리지 시스템을 변경해야 할 수 있습니다.

"백엔드 구성"

Astra Trident 포트

Astra Trident는 통신을 위해 특정 포트에 액세스해야 합니다.

"Astra Trident 포트"

컨테이너 이미지 및 해당 Kubernetes 버전

공기 박형 설치의 경우 다음 목록은 Astra Trident를 설치하는 데 필요한 컨테이너 이미지의 참조입니다. 를 사용합니다 `tridentctl images` 명령을 사용하여 필요한 컨테이너 이미지 목록을 확인합니다.

Kubernetes 버전	컨테이너 이미지
v1.22.0	• Docker.IO/NetApp/트라이덴트:23.07.1 • Docker.IO/netapp/trident-autosupport:23.07 • registry.k8s.io/sig-storage/csi-provisioner: v3.5.0 • 레지스트리.k8s.io/sig-storage/csi-attacher:v4.3.0 • registry.k8s.io/sig-storage/csi-resizer:v1.8.0 • 레지스트리.k8s.io/sig-storage/csi-shotter:v6.2.2 • registry.k8s.io/sig-storage/csi-node-driver-register:v2.8.0 • Docker.IO/netapp/trident-operator:23.07.1 (선택사항)
v1.23.0	• Docker.IO/NetApp/트라이덴트:23.07.1 • Docker.IO/netapp/trident-autosupport:23.07 • registry.k8s.io/sig-storage/csi-provisioner: v3.5.0 • 레지스트리.k8s.io/sig-storage/csi-attacher:v4.3.0 • registry.k8s.io/sig-storage/csi-resizer:v1.8.0 • 레지스트리.k8s.io/sig-storage/csi-shotter:v6.2.2 • registry.k8s.io/sig-storage/csi-node-driver-register:v2.8.0 • Docker.IO/netapp/trident-operator:23.07.1 (선택사항)

Kubernetes 버전	컨테이너 이미지
v1.24.0	• Docker.IO/NetApp/트라이덴트:23.07.1 • Docker.IO/netapp/trident-autosupport:23.07 • registry.k8s.io/sig-storage/csi-provisioner: v3.5.0 • 레지스트리.k8s.io/sig-storage/csi-attacher:v4.3.0 • registry.k8s.io/sig-storage/csi-resizer:v1.8.0 • 레지스트리.k8s.io/sig-storage/csi-shotter:v6.2.2 • registry.k8s.io/sig-storage/csi-node-driver-register:v2.8.0 • Docker.IO/netapp/trident-operator:23.07.1 (선택사항)
v1.25.0	• Docker.IO/NetApp/트라이덴트:23.07.1 • Docker.IO/netapp/trident-autosupport:23.07 • registry.k8s.io/sig-storage/csi-provisioner: v3.5.0 • 레지스트리.k8s.io/sig-storage/csi-attacher:v4.3.0 • registry.k8s.io/sig-storage/csi-resizer:v1.8.0 • 레지스트리.k8s.io/sig-storage/csi-shotter:v6.2.2 • registry.k8s.io/sig-storage/csi-node-driver-register:v2.8.0 • Docker.IO/netapp/trident-operator:23.07.1 (선택사항)
v1.26.0	• Docker.IO/NetApp/트라이덴트:23.07.1 • Docker.IO/netapp/trident-autosupport:23.07 • registry.k8s.io/sig-storage/csi-provisioner: v3.5.0 • 레지스트리.k8s.io/sig-storage/csi-attacher:v4.3.0 • registry.k8s.io/sig-storage/csi-resizer:v1.8.0 • 레지스트리.k8s.io/sig-storage/csi-shotter:v6.2.2 • registry.k8s.io/sig-storage/csi-node-driver-register:v2.8.0 • Docker.IO/netapp/trident-operator:23.07.1 (선택사항)

Kubernetes 버전	컨테이너 이미지
v1.27.0	• Docker.IO/NetApp/트라이덴트:23.07.1 • Docker.IO/netapp/trident-autosupport:23.07 • registry.k8s.io/sig-storage/csi-provisioner: v3.5.0 • 레지스트리.k8s.io/sig-storage/csi-attacher:v4.3.0 • registry.k8s.io/sig-storage/csi-resizer:v1.8.0 • 레지스트리.k8s.io/sig-storage/csi-shotter:v6.2.2 • registry.k8s.io/sig-storage/csi-node-driver-register:v2.8.0 • Docker.IO/netapp/trident-operator:23.07.1 (선택 사항)

저작권 정보

Copyright © 2023 NetApp, Inc. All Rights Reserved. 미국에서 인쇄된 본 문서의 어떠한 부분도 저작권 소유자의 사전 서면 승인 없이는 어떠한 형식이나 수단(복사, 녹음, 녹화 또는 전자 검색 시스템에 저장하는 것을 비롯한 그래픽, 전자적 또는 기계적 방법)으로도 복제될 수 없습니다.

NetApp이 저작권을 가진 자료에 있는 소프트웨어에는 아래의 라이센스와 고지사항이 적용됩니다.

본 소프트웨어는 NetApp에 의해 '있는 그대로' 제공되며 상품성 및 특정 목적에의 적합성에 대한 명시적 또는 묵시적 보증을 포함하여(이에 제한되지 않음) 어떠한 보증도 하지 않습니다. NetApp은 대체품 또는 대체 서비스의 조달, 사용 불능, 데이터 손실, 이익 손실, 영업 중단을 포함하여(이에 국한되지 않음), 이 소프트웨어의 사용으로 인해 발생하는 모든 직접 및 간접 손해, 우발적 손해, 특별 손해, 징벌적 손해, 결과적 손해의 발생에 대하여 그 발생 이유, 책임론, 계약 여부, 엄격한 책임, 불법 행위(과실 또는 그렇지 않은 경우)와 관계없이 어떠한 책임도 지지 않으며, 이와 같은 손실의 발생 가능성이 통지되었다 하더라도 마찬가지입니다.

NetApp은 본 문서에 설명된 제품을 언제든지 예고 없이 변경할 권리를 보유합니다. NetApp은 NetApp의 명시적인 서면 동의를 받은 경우를 제외하고 본 문서에 설명된 제품을 사용하여 발생하는 어떠한 문제에도 책임을 지지 않습니다. 본 제품의 사용 또는 구매의 경우 NetApp에서는 어떠한 특허권, 상표권 또는 기타 지적 재산권이 적용되는 라이센스도 제공하지 않습니다.

본 설명서에 설명된 제품은 하나 이상의 미국 특허, 해외 특허 또는 출원 중인 특허로 보호됩니다.

제한적 권리 표시: 정부에 의한 사용, 복제 또는 공개에는 DFARS 252.227-7013(2014년 2월) 및 FAR 52.227-19(2007년 12월)의 기술 데이터-비상업적 품목에 대한 권리(Rights in Technical Data -Noncommercial Items) 조항의 하위 조항 (b)(3)에 설명된 제한사항이 적용됩니다.

여기에 포함된 데이터는 상업용 제품 및/또는 상업용 서비스(FAR 2.101에 정의)에 해당하며 NetApp, Inc.의 독점 자산입니다. 본 계약에 따라 제공되는 모든 NetApp 기술 데이터 및 컴퓨터 소프트웨어는 본질적으로 상업용이며 개인 비용만으로 개발되었습니다. 미국 정부는 데이터가 제공된 미국 계약과 관련하여 해당 계약을 지원하는 데에만 데이터에 대한 전 세계적으로 비독점적이고 양도할 수 없으며 재사용이 불가능하며 취소 불가능한 라이센스를 제한적으로 가집니다. 여기에 제공된 경우를 제외하고 NetApp, Inc.의 사전 서면 승인 없이는 이 데이터를 사용, 공개, 재생산, 수정, 수행 또는 표시할 수 없습니다. 미국 국방부에 대한 정부 라이센스는 DFARS 조항 252.227-7015(b)(2014년 2월)에 명시된 권한으로 제한됩니다.

상표 정보

NETAPP, NETAPP 로고 및 http://www.netapp.com/TM에 나열된 마크는 NetApp, Inc.의 상표입니다. 기타 회사 및 제품 이름은 해당 소유자의 상표일 수 있습니다.